PROG3271 – Open Source Web Programming

Milestone 2

Prepared by:

David Jaiyeola

Stryfe Vidmar

Revienne Galang

Instructor:

Angelina Ziesemer

Project Description

Sproutly is designed with one goal in mind: to help people take better care of their plants. To do this, our users need top-notch plant care, and by using our app, they can more easily and effectively reach their goals. We allow users to keep track of their houseplants by providing all the details they need and setting up their own plant care reminders. We also have a vast catalog of plants that users can search through to learn or identify certain types of plants and their requirements.

Database Design

Users - stores user account information for authentication and account management

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique user identifier
username	VARCHAR(50)	user's username
email	VARCHAR(100)	User's email address
password_hash	VARCHAR(255)	Encrypted password
created_at	TIMESTAMP	Account creation date

Relationships

- One user can own many UserPlants(1:M)
- One user can have many favorites(1:M)

Plant Catalog - stores information about plant species available for browsing and identification

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique plant identifier
common_name	VARCHAR(100)	Common plant name
scientific_name	VARCHAR(100)	Scientific plant name
description	TEXT	Plant description
watering_guide	TEXT	Watering instruction
sunlight_req	VARCHAR(50)	Sunlight requirement
image_url	VARCHAR(255)	Plant image URL

Relationships

- ∉ One PlantCatalog record can be linked to many UserPlants (1:M)
- ∉ One PlantCatalog record can appear in many Favorites (1:M)
- ∉ One PlantCatalog record can appear in many StoreInventory records (1:M)

User Plants - Stores plants Owned by users and supports CRUD operations.

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique plant instance identifier
user_id	INT(FK)	Reference users
catalog_id	INT(FK)	Reference PlantCatalog
custom name	VARCHAR(100)	User defined plant name
location	ENUM(LR, Ki, BR, B)	Plant location
last watered	DATE	Last watering date
date added	TIMESTAMP	Date plant was added

Relationships

- ∉ Many UserPlants belong to one User (M:1)
- ∉ Many UserPlants may reference one PlantCatalog record (M:1)
- ∉ One UserPlant can have many Reminders (1:M)

Reminders - stores care reminders for user plants

Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique reminder identifier
user_plant_id	INT(FK)	Reference userPlants
reminder_type	ENUM	type of reminder
frequency_days	INT	Reminder frequency
next_due_date	DATE	Next scheduled reminder
is_completed	BOOLEAN	Completion status

Relationships

- ∉ Many Reminders belong to one UserPlant (M:1)

Favorites - stores plants that user has marked as favorite

Fields

Field	Datatype	Description
user_id	INT(PK, FK)	Reference Users
catalog_id	INT(PK, FK)	Reference PlantCatalog

Relationships

- ∉ Many Favorites belong to one User (M:1)
- ∉ Many Favorites belong to one PlantCatalog record (M:1)
- ∉ Creates a Many-to-Many relationship between Users and PlantCatalog

Stores - stores information about local plant stores

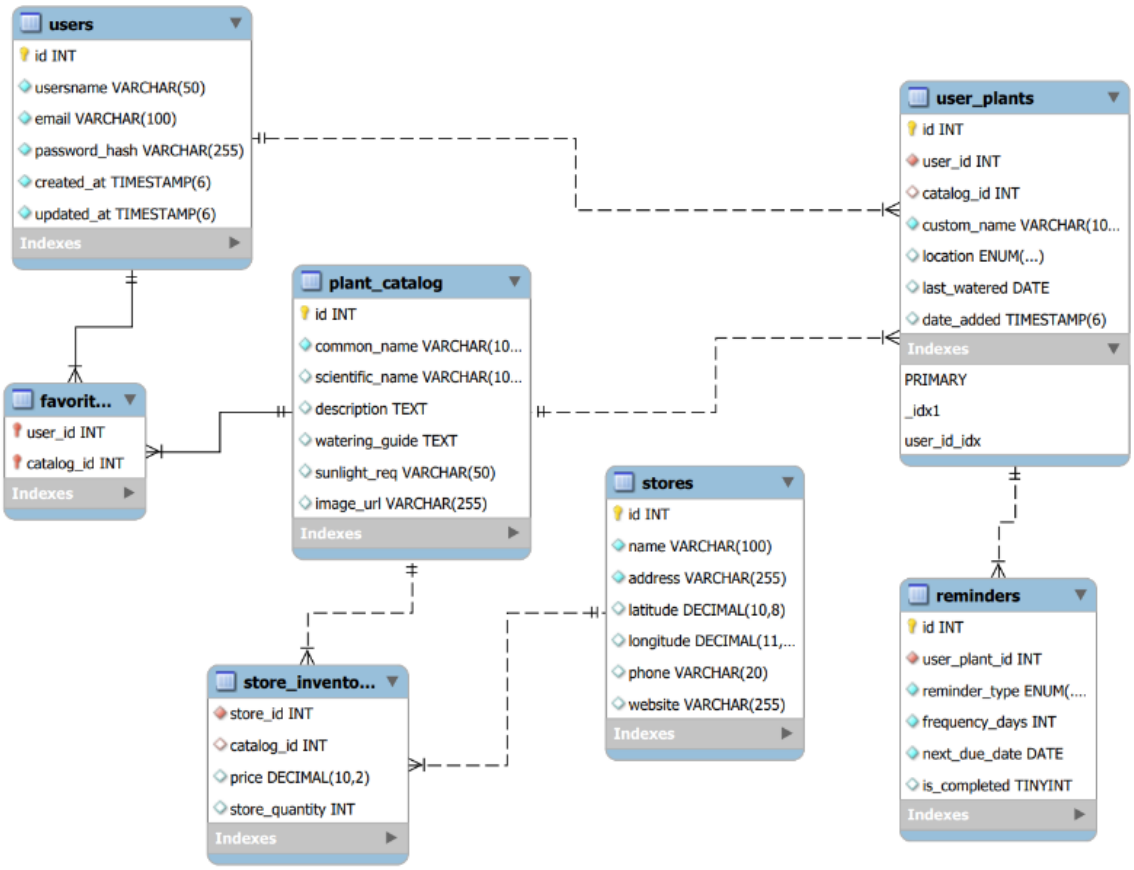
Fields

Field	Datatype	Description
id	INT(PK, AUTO_INCREMENT)	Unique store identifier
name	VARCHAR(100)	store name
address	VARCHAR(255)	store address
latitude	DECIMAL(10, 8)	Geographic latitude
longtitude	DECIMAL(11, 8)	Geographic longtitude
phone	VARCHAR(20)	Contact number
website	VARCHAR(255)	store website URL

Relationships

- ∉ One Store can have many StoreInventory records (1:M)

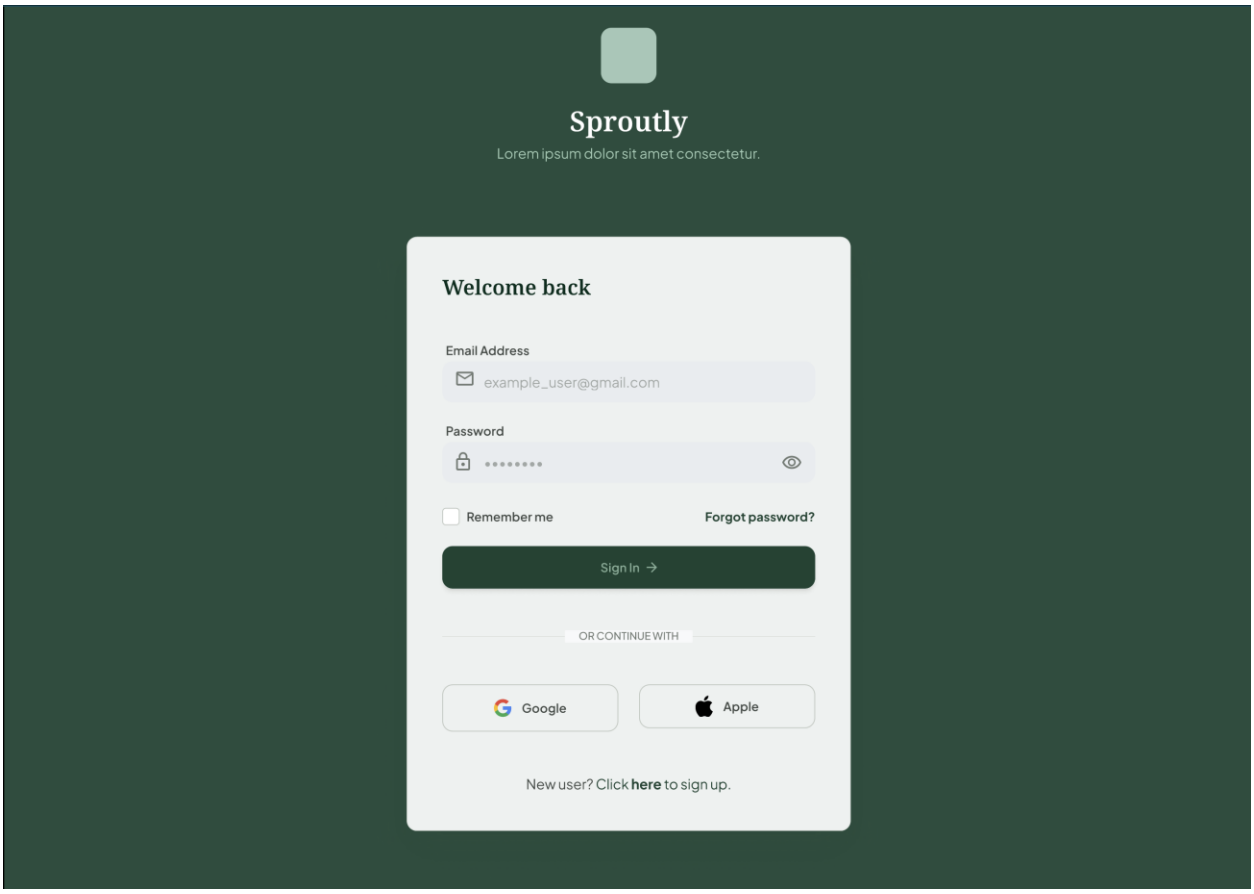
ERD/Database Diagram



Frontend Wireframes

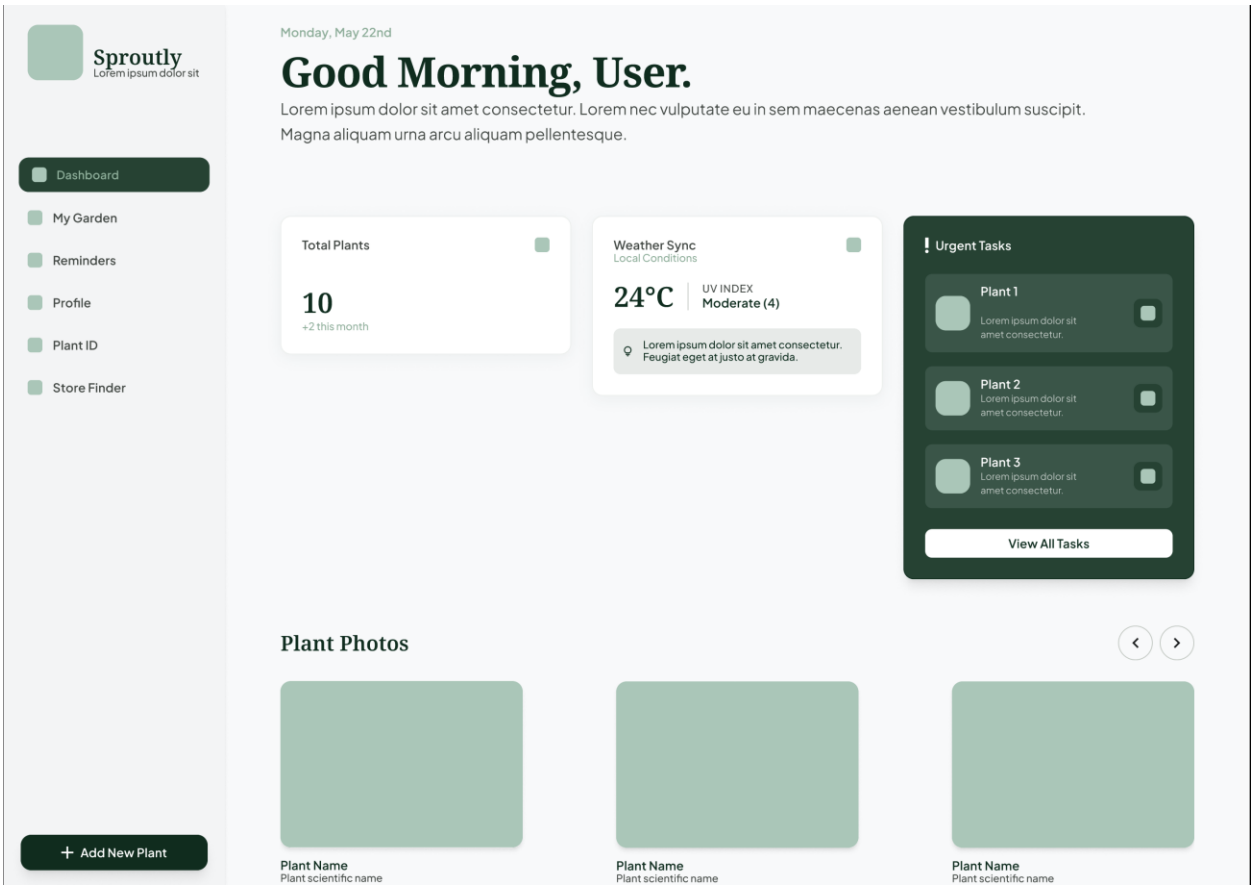
Design Philosophy – our design focuses on simplicity, intuitive navigation, and clean layout

Login screen



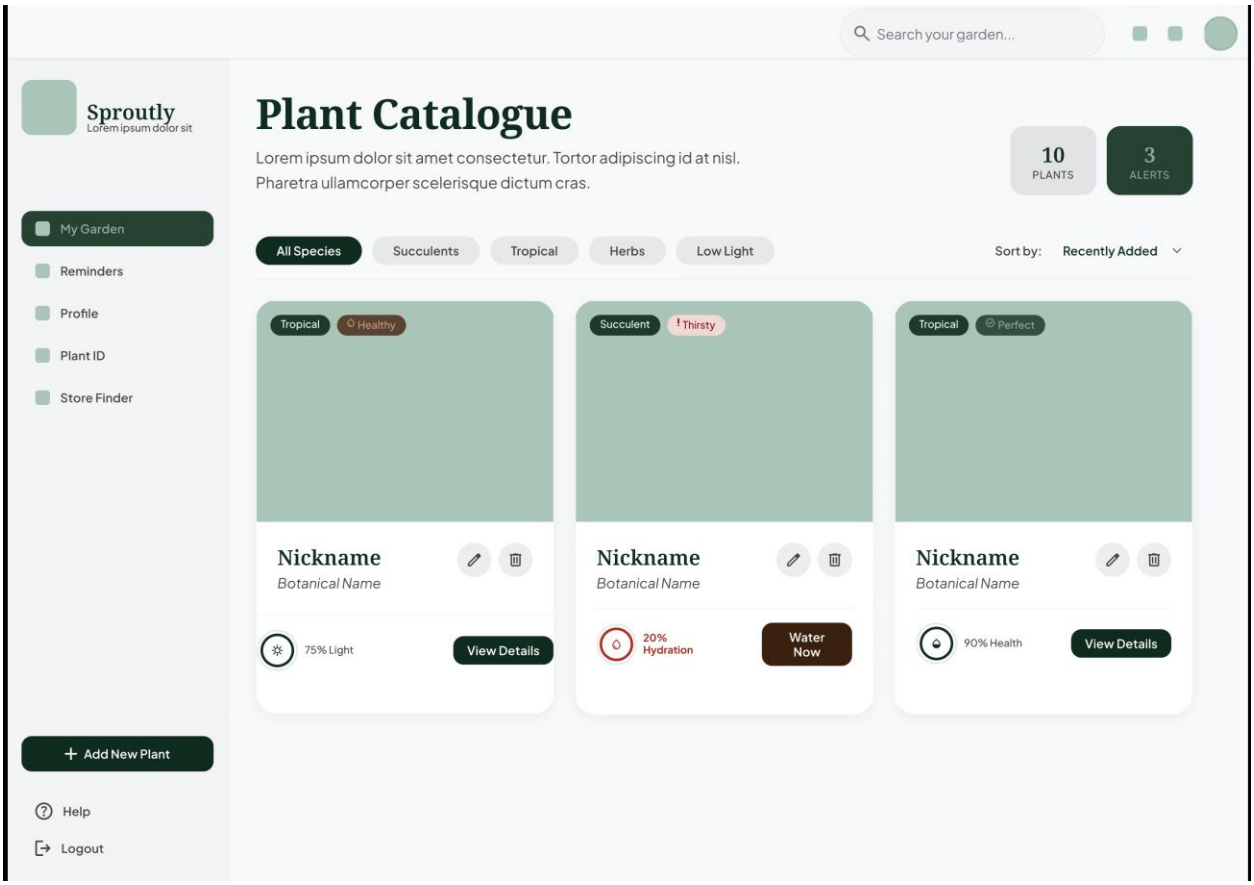
Dashboard – This screen displays what users can see once logged in

This page serves as an immediate plant status checker. Instead of the user going to the “My Garden” page, users can instantly see the high-level status of the plants

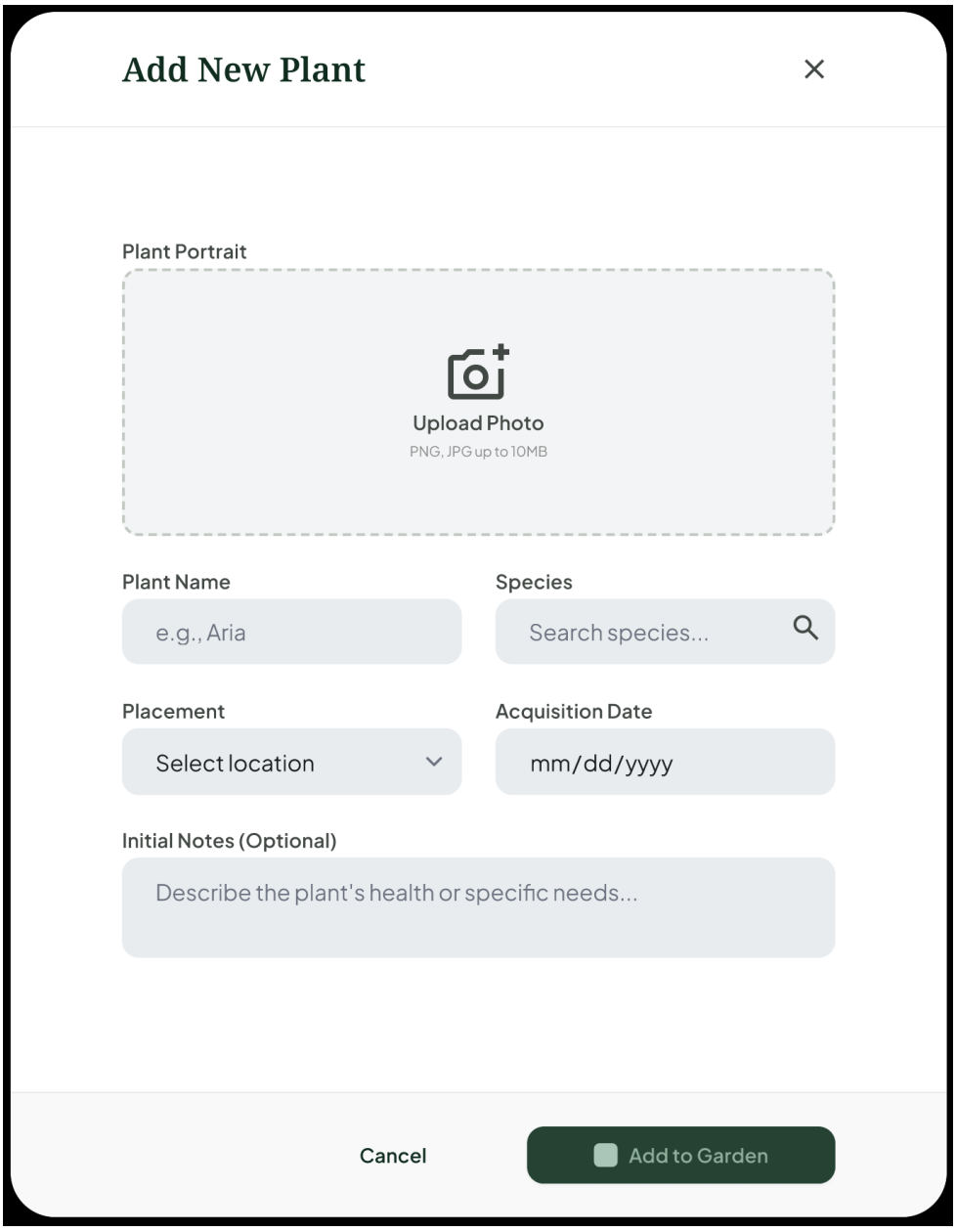


Plant Catalogue – This page displays the plants added by the user

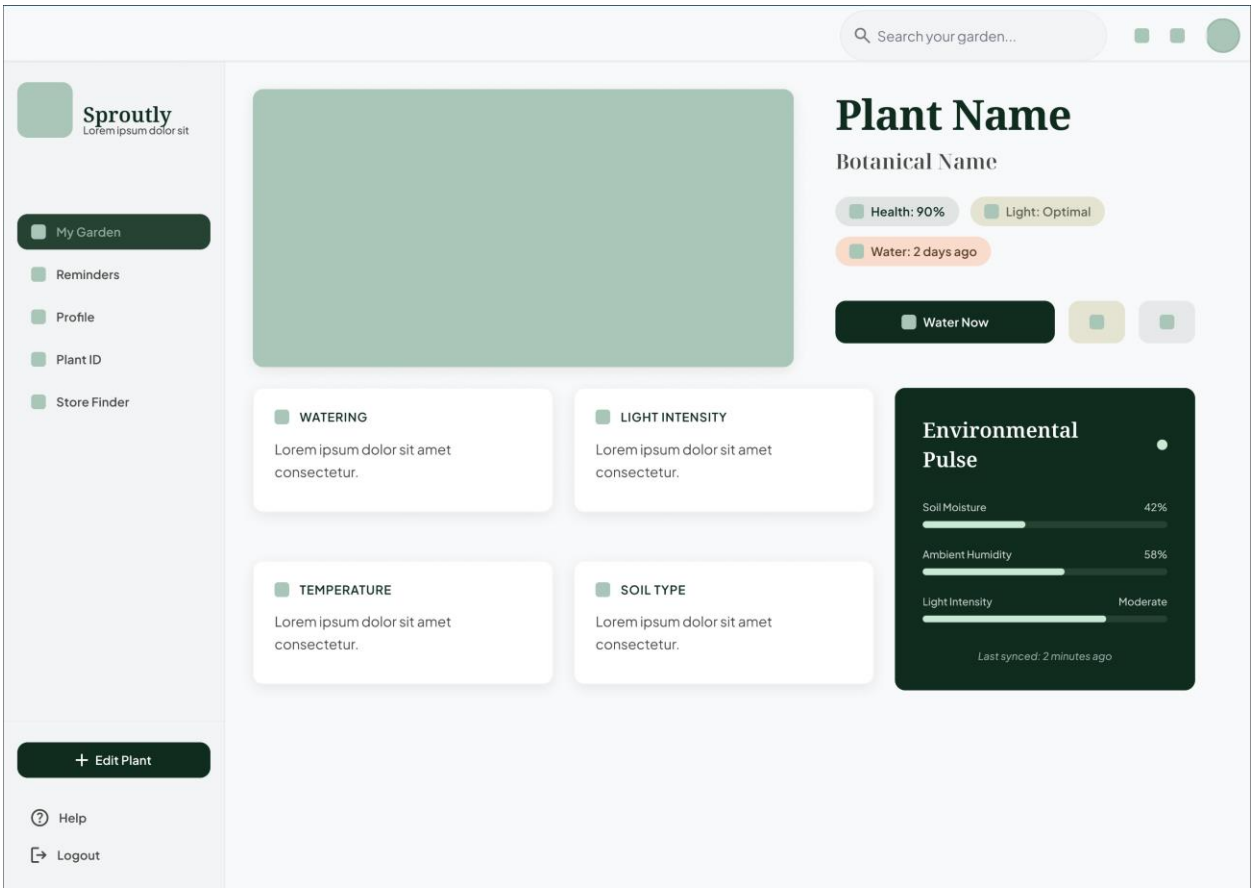
In this page, we use cards to display the plants owned by the user. Users can immediately see the plant information like plant name, botanical name, and health status.



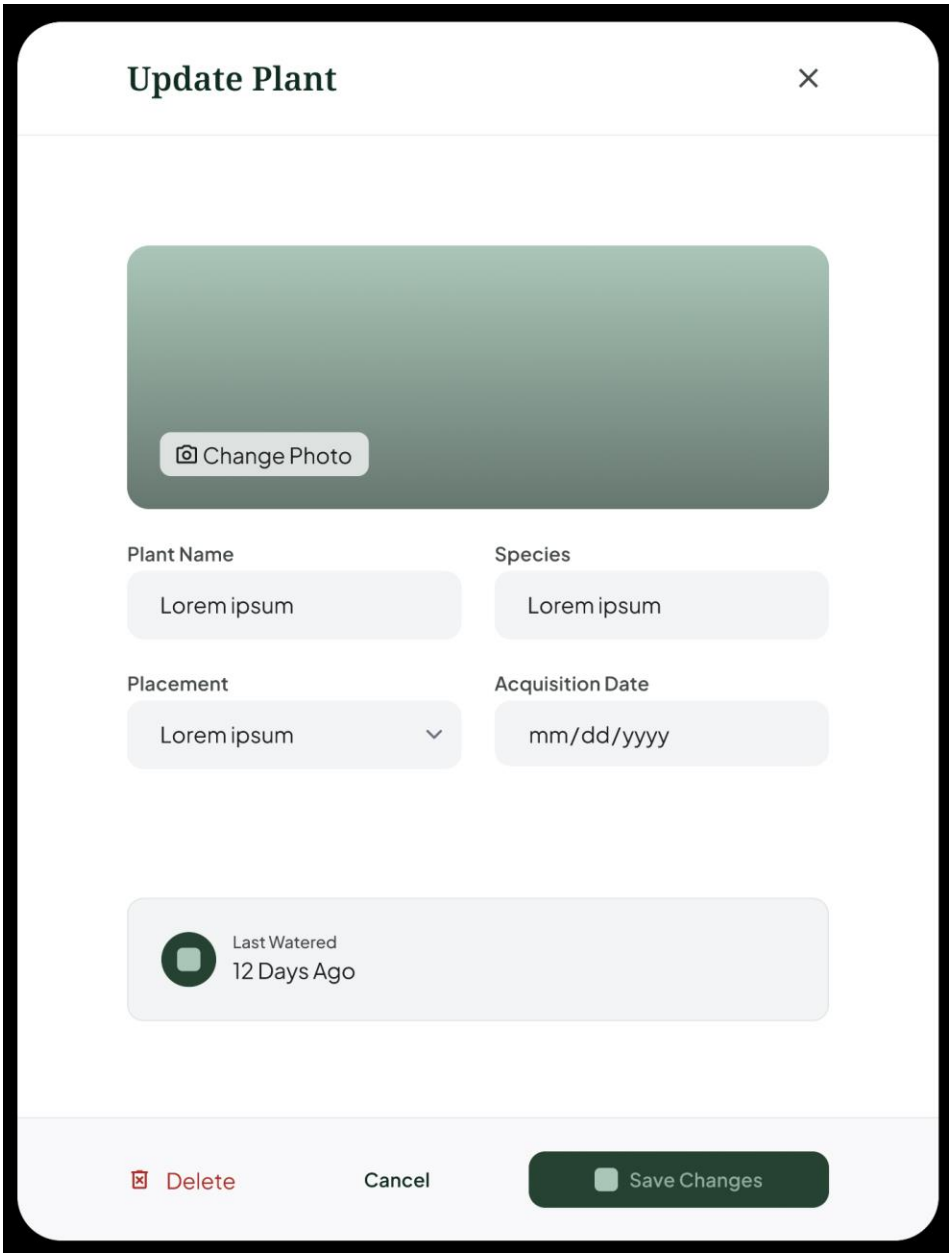
Add New Plant – Modal pop-up to add new plant to the user’s catalogue



Plant Details – This page displays the plant information

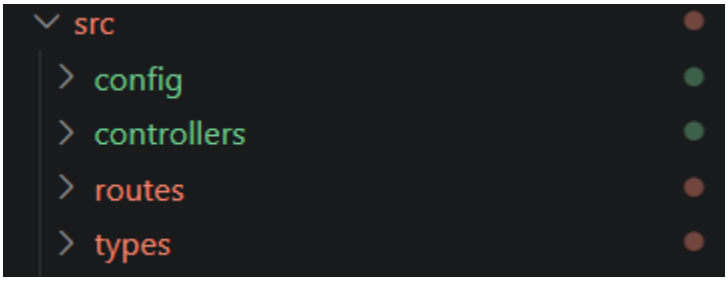


Update/Delete Plant – Modal pop-up that allows the user to edit or delete plant



Planned API routes/Backend

Configuring all our API routes in our SRC folder, containing config (database config), controllers (response handling), routes (holding express routes), types (contains all our model types). Of course, none of this is currently connected to our database, so many errors are showing up in our placeholder data, but it is a good demonstration of our route handling.



Config

Our MySQL database connection contains our database pool for now, for the purpose of pre-established cached connections.

```
src > config > TS database.ts > ...
1   // Database configuration file for mySQL
2   import mysql from 'mysql2/promise';
3   import dotenv from 'dotenv';
4
5   // Temporary placeholder database config
6   dotenv.config();
7
8   const pool = mysql.createPool({
9       host: process.env.DB_HOST,
10      user: process.env.DB_USER,
11      password: process.env.DB_PASSWORD,
12      database: process.env.DB_NAME,
13      waitForConnections: true,
14      connectionLimit: 10,
15      queueLimit: 0
16  });
```

Controllers

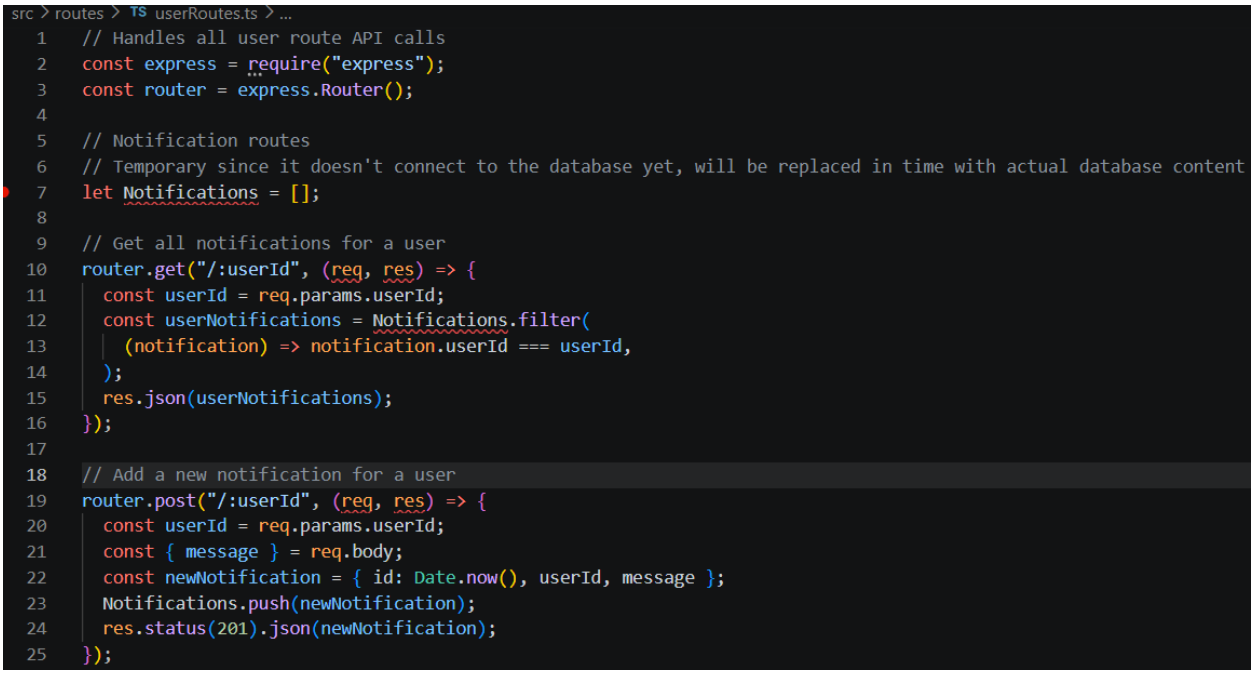
Our user controller, which handles all the CRUD operations pertaining to our user model, we've only added the GET request for now for demonstration purposes. We also will have a controller for our catalog, store finder, etc.

src > controllers > TS userControllers.ts > ...

```ts
// Temporary user controllers for demonstrations
import { Request, Response } from "express";

// Get user information
export const getUserInfo = (req: Request, res: Response) => {
  const userId = req.params.userId;
  // Temporary user data
    const user = {
    id: userId,
    name: "John Doe",
    email: "John@gmail.com",
    password: "password123",
  };
  res.json(user);
    }
```

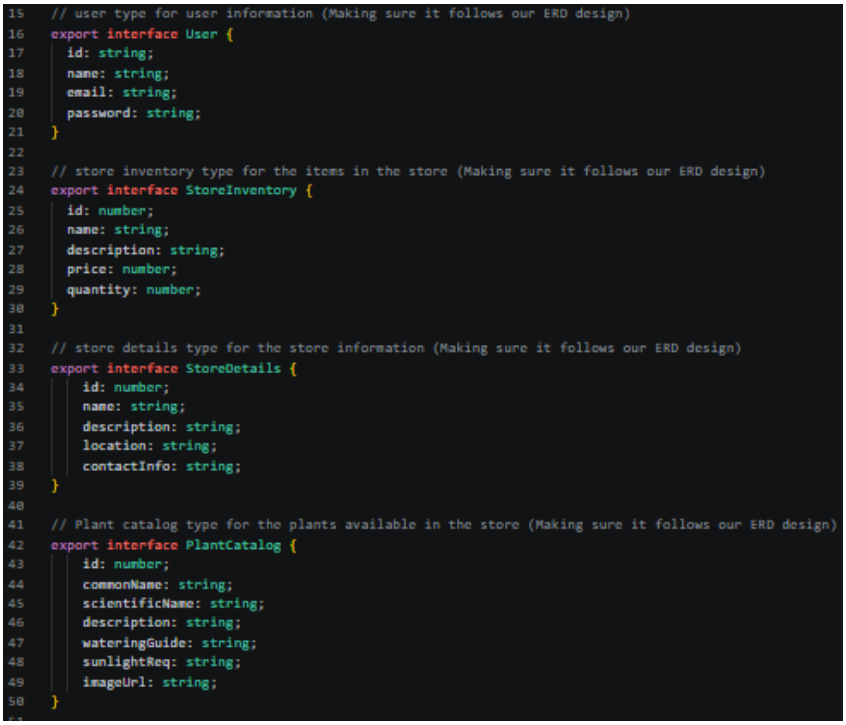
Routes

These are all the routes for our current application. Each handles its own API calls for the user, e.g., get user notifications and add new user notifications.



Types

These are most of our types, all relating to the user and heavily based on our ERD diagram.



```
// user type for user information (Making sure it follows our ERD design)
export interface User {
  id: string;
  name: string;
  email: string;
  password: string;
}

// store inventory type for the items in the store (Making sure it follows our ERD design)
export interface StoreInventory {
  id: number;
  name: string;
  description: string;
  price: number;
  quantity: number;
}

// store details type for the store information (Making sure it follows our ERD design)
export interface StoreDetails {
    id: number;
    name: string;
    description: string;
    location: string;
    contactInfo: string;
}

// Plant catalog type for the plants available in the store (Making sure it follows our ERD design)
export interface PlantCatalog {
    id: number;
    commonName: string;
    scientificName: string;
    description: string;
    wateringGuide: string;
    sunlightReq: string;
    imageUrl: string;
}
```

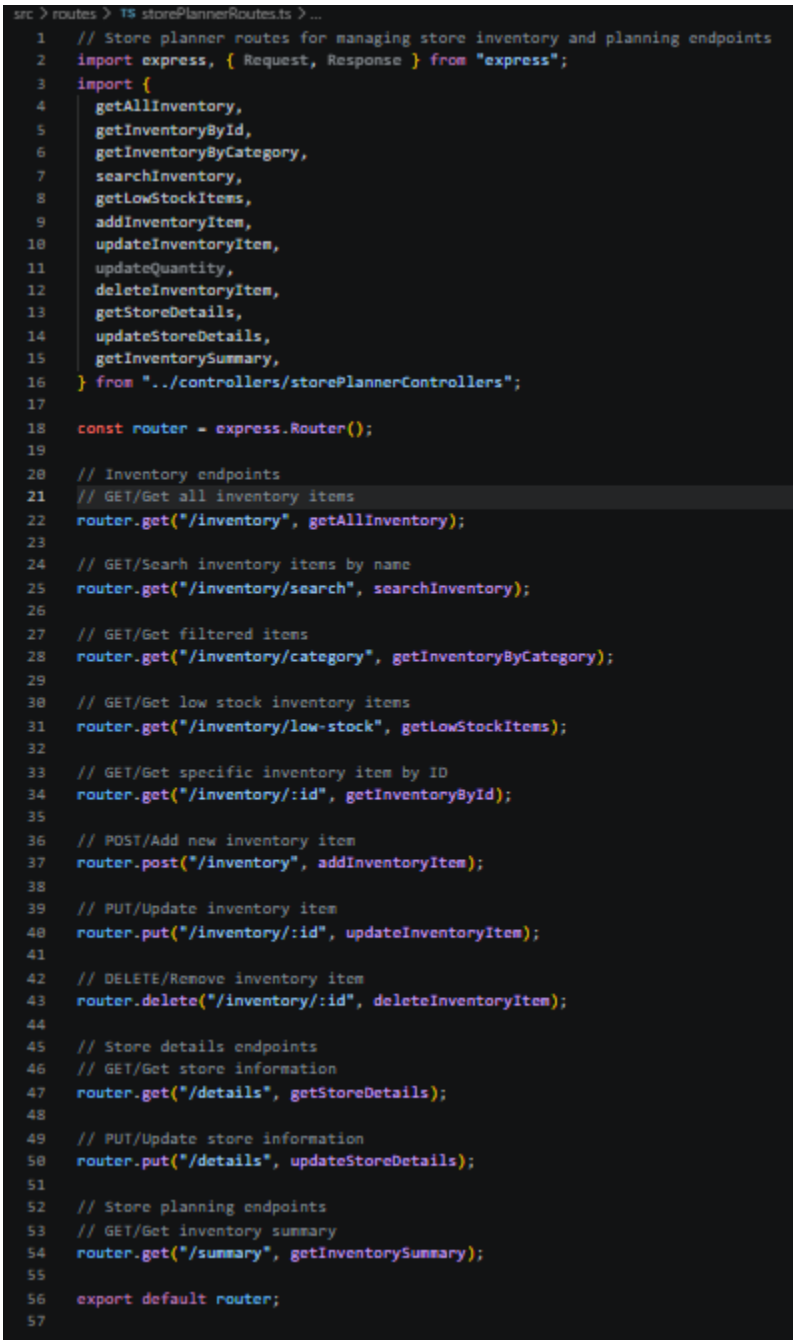
API Endpoints

Plant Catalog Endpoints

src > routes > TS plantCatalogRoutes.ts > ...

```ts
// Plant catalog routes for managing plant information endpoints
import express, { Request, Response } from "express";
import {
  getAllPlants,
  getPlantById,
  searchPlants,
  filterByLightRequirements,
  addPlant,
  updatePlant,
  deletePlant,
} from "../controllers/plantCatalogControllers";

const router = express.Router();

// GET/Get all plants in catalog
router.get("/", getAllPlants);

// GET/Search plants in catalog by name
router.get("/search", searchPlants);

// GET/Filter plants in catalog by sunlight requirements
router.get("/light", filterByLightRequirements);

// GET/Get plant by ID
router.get("/:id", getPlantById);

// POST/Add plant to catalog
router.post("/", addPlant);

// PUT/Update plant in catalog
router.put("/:id", updatePlant);

// DELETE/Remove plant from catalog
router.delete("/:id", deletePlant);

export default router;
```

Inventory Endpoints



Both endpoints will be connected to a full-fledged controller for each page to properly handle all their CRUD operations in full.

Team Responsibilities

David Jaiyeola – Database Designer

Stryfe Vidmar – Backend Planner

Revienne Galang – Frontend Designer